Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ingeniería Eléctrica

Tarea N°4

Integrantes: Diego Vega
Profesor: Javier Ruiz del Solar
Auxiliar: Patricio Loncomilla
Ayudantes: Benjamín Llancao Mella
Bruno Cavieres L.
Gonzalo Olguín M.

Fecha de realización: 20 de junio de 2024
Fecha de entrega: 16 de Junio de 2024
Santiago de Chile

Introducción

En el presente informe detalla el desarrollo de la tarea N°4 del curso de Inteligencia Computacional. El desarrollo de la tarea se hizo acorde a las indicaciones del documento *Tarea 4 EL4106 2024 v04-JRS.pdf*.
En el documento mencionado anteriormente, se tiene por objetivo el estudio e implementación de redes neuronales en un problema de clasificación de dígitos. Se utilizara el conjunto de datos *Optical Recognition of Handwritten Digits Data Set*, el cual fue entregado por el cuerpo docente en los archivos *1_digits_train.txt* y *1_digits_test.txt*. El conjunto posee 64 características, 10 clases y 5620 muestras en total.
Las redes neuronales a entrenadas deben una capa de entrada de dimensionalidad 64, 1 o 2 capas ocultas de dimensionalidad por definir y una capa de salida de dimensionalidad 10. La función *loss* es entropía cruzada. El optimizador es *Adam*. La función *softmax* esta implícita en la función de perdida utilizada. El minibatch sera de 128. La tasa de aprendizaje sera 0.001. Además, para evitar el *overfitting* se implemento un enfoque de *patience early stopping*. Los objetivos de la tara, y por tanto lo realizado, consisten en:

Ejecución del código base Esta sección consiste en: la ejecución del código base entregado por el cuerpo docente. Dicho código carga los archivos del Dataset en dos DataFrames, los cuales son utilizados para generar tres DataFrames, entrenamiento, validación y prueba. Luego estos DataFrames son utilizados para generar DataLoaders, los cuales permiten acceder a los elementos de los DataFrames mediante batches. Además, los datos fueron normalizados. Finalmente, se entrena un red neuronal de ejemplo, en la cual no esta implementado el *patience early stopping*.

Entrenamiento y métricas de varias redes neuronales Esta sección consiste en: El entrenamiento de 6 redes neuronales distintas, en las cuales se implementa el enfoque de *patience early stopping* para detener el entrenamiento previo al *overfitting*. Además. para cada red entrenada: Se calculo el *loss* de entrenamiento y validación, el tiempo total de entrenamiento, se gráfico el *loss* de entrenamiento y validación en función del número de épocas, se genero la matriz de confusión normalizada y el accuracy utilizando el conjunto de entrenamiento y el de validación.

Métricas de la mejor red neuronal Esta sección consiste en: la elección de la red con mayor accuracy en validación y el calculo de la matriz de confusión normalizada y el accuracy normalizado utilizando el conjunto de prueba.

Análisis Esta sección consiste en: La explicación de los efectos de variar la cantidad de neuronas en la capa oculta y cómo esto afecta el desempeño de la red. La explicación de variar la cantidad de capas ocultas y como esto afecta el desempeño de la red. La explicación del efecto de la función de activación y cómo esto afecta el desempeño de la red. El análisis de los tiempos de entrenamiento, matrices de confusión y accuracies de las arquitecturas probadas en validación. El análisis de la matriz de confusión y el accuracy en el conjunto de prueba, en comparación a los obtenidos en validación.

Índice de Contenidos

1. Desarrollo **1**
1.1. 1) . . . 1
1.2. 2) . . . 1
1.2.1. a) . . . 5
1.2.2. b) . . . 8
1.2.3. c) . . . 11
1.2.4. d) . . . 14
1.2.5. e) . . . 17
1.2.6. f) . . . 20
1.3. 3) . . . 23
1.4. 4) . . . 24
1.4.1. a) . . . 24
1.4.2. b) . . . 25
1.4.3. c) . . . 26
1.4.4. d) . . . 27
1.4.5. e) . . . 28

2. Conclusiones **29**
2.1. Aprendizajes . . . 29
2.2. Dificultades . . . 29
2.3. Evaluación de resultados . . . 29

Anexo A. Código **30**

Índice de Figuras

1. Gráfico perdidas de entrenamiento y validación versus época - Primera red neuronal. . . . 6
2. Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Primera red neuronal. . . . 7
3. Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Primera red neuronal. . . . 8
4. Gráfico perdidas de entrenamiento y validación versus época - Segunda red neuronal. . . . 9
5. Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Segunda red neuronal. . . . 10
6. Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Segunda red neuronal. . . . 11
7. Gráfico perdidas de entrenamiento y validación versus época - Tercera red neuronal. . . . 12

8. Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Tercera red neuronal. . . . 13
9. Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Tercera red neuronal. . . . 14
10. Gráfico perdidas de entrenamiento y validación versus época - Cuarta red neuronal. . . . 15
11. Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Cuarta red neuronal. . . . 16
12. Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Cuarta red neuronal. . . . 17
13. Gráfico perdidas de entrenamiento y validación versus época - Quinta red neuronal. . . . 18
14. Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Quinta red neuronal. . . . 19
15. Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Quinta red neuronal. . . . 20
16. Gráfico perdidas de entrenamiento y validación versus época - Sexta red neuronal. . . . 21
17. Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Sexta red neuronal. . . . 22
18. Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Sexta red neuronal. . . . 23
19. Matriz de confusión normalizada y accuracy normalizado sobre conjunto de prueba - Primera red neuronal. . . . 24

Índice de Tablas

1. Arquitectura de la primera red neuronal. . . . 5
2. Arquitectura de la segunda red neuronal. . . . 8
3. Arquitectura de la tercera red neuronal. . . . 11
4. Arquitectura de la cuarta red neuronal. . . . 14
5. Arquitectura de la quinta red neuronal. . . . 18
6. Arquitectura de la sexta red neuronal. . . . 21
7. Tiempos de entrenamiento de los modelos. . . . 27

Índice de Códigos

1. Clase *Model*. . . . 2
2. Función *ColorSelector*. . . . 3
3. Función *TrainModel*. . . . 3
4. Red neuronal con 10 neuronas en la capa oculta y función de activación *ReLU*. 5
5. Red neuronal con 40 neuronas en la capa oculta y función de activación *ReLU*. 8

6. Red neuronal con 10 neuronas en la capa oculta y función de activación *Tanh*. 11
7. Red neuronal con 40 neuronas en la capa oculta y función de activación *Tanh*. 14
8. Red neuronal con 10 neuronas en la capa oculta, 2 capas ocultas y función de activación *ReLU*. 17
9. Red neuronal con 40 neuronas en la capa oculta, 2 capas ocultas y función de activación *ReLU*. 20
10. Función *EvaluateBestModel* 23
A.1. Código de la tarea. 30

1. Desarrollo

1.1. 1)

En la presente sección se ejecuto el código base, el cual consiste de 5 partes, las cuales se describen a continuación.

Subir Dataset de digits Esta sección consiste en: Subir los archivos mencionados previamente al entorno de Google Colaboratory.

Leer Dataset de digits Esta sección consiste en: La lectura y manejo de los Datasets subidos en la sección previa. Durante la lectura se agregan las características por medio de las columnas del DataFrame. Además, el manejo de los Datasets consiste en la división en los tres conjuntos (entrenamiento, validación y prueba) y en el escalado de estos mediante *StandarScaler*.

Crear Datasets y DataLoaders para pytorch Esta sección consiste en: La creación de los Datasets para los 3 conjuntos, estos Datasets consisten en arreglos de *numpy* que contienen lo que previamente contenían los DataFrames de los 3 conjuntos. Posteriormente, se crean los DataLoaders de los 3 conjuntos a partir de los Datasets definidos previamente, los Dataloaders son utilizados para el entrenamiento de las redes neuronales en *Pytorch*. Por ultimo, es importante señalar que durante la creación de los DataLoaders se utiliza un *batchsize* de 128 y un parámetro *shuffle* en True.

Crear modelo Esta sección consiste en: La creación de un modelo de varias capas utilizando *torch.nn.Sequential*. Posterior a la creación del modelo, se define el *device* a *cuda*, para que el entrenamiento de las redes sea más rápido, entre otras cosas, luego se «manda» el modelo al *device*. Finalmente se define la función de perdida como *torch.nn.CrossEntropyLoss* y se define el optimizador como *torch.optim.Adam*, con un *learning rate* de 0.001. Estos hiper-parámetros son utilizados en todas las redes neuronales implementadas.

Entrenamiento Esta sección consiste en: El entrenamiento de la red neuronal definida en la sección anterior. Este entrenamiento no posee *patience early stopping* y no brinda ninguna métrica de evaluación del modelo, por lo que el código entregado es solo una base para la implementación solicitada en 1.2.

1.2. 2)

En la presente sección se entrenaron diversas redes neuronales, las cuales poseen diferentes arquitecturas entre ellas, es decir, diferente cantidad de neuronas ocultas, capas ocultas y funciones de activación.
Además, de cada red se solicita: Calcular la perdida sobre el conjunto de entrenamiento y sobre el conjunto de validación, graficar la perdida sobre el conjunto de entrenamiento y sobre el conjunto de validación en función de las épocas, generar la matriz de confusión

normalizada y el accuracy sobre el conjunto de entrenamiento previo al early stopping y generar la matriz de confusión normalizada y el accuracy normalizado sobre el conjunto de validación. Es importante señalar que las matrices de confusión tendran distintos colores dependiendo del valor del accuracy.
Para realizar lo solicitado se definió una clase de python y dos funciones de python.
La clase implementada se llama *Model* y hereda atributos de *torch.nn.Module* y permite la creación de modelos de redes neuronales de forma sencilla. Como parámetros recibe un string que indica el nombre de la función de activación de la capa oculta, un int que indica la cantidad de neuronas ocultas y un int que indica la cantidad de capas ocultas. El método foward es el encargado de concatenar las capas de la red neuronal creada previamente, dicho método es propio de *torch.nn.Module*. El código de la clase se adjunta en 1.

Código 1: Clase *Model*.

```
class Model(nn.Module):
  def __init__(self, activation_layer, hidden_size, n_hidden_layers, input_size=64,
    ↪ output_size=10):
    super(Model, self).__init__()
    self.input_layer = nn.Linear(input_size, hidden_size)
    if n_hidden_layers == 2:
      self.hidden_layer = nn.Sequential(nn.ReLU() if activation_layer == 'ReLU' else nn.
     ↪ Tanh() if activation_layer == 'Tanh' else None,
                                        nn.ReLU() if activation_layer == 'ReLU' else nn.Tanh() if
     ↪ activation_layer == 'Tanh' else None)
    elif n_hidden_layers == 1:
      self.hidden_layer = nn.ReLU() if activation_layer == 'ReLU' else nn.Tanh() if
     ↪ activation_layer == 'Tanh' else None
    self.output_layer = nn.Linear(hidden_size, output_size)
    self.n_hidden_layers = n_hidden_layers

  def forward(self, x):
    out = self.input_layer(x)
    if self.hidden_layer is not None:
      if self.n_hidden_layers == 2:
        for layer in self.hidden_layer:
          out = layer(out)
      elif self.n_hidden_layers == 1:
        out = self.hidden_layer(out)
    out = self.output_layer(out)
    return out
```

La primera función implementada se llama *ColorSelector* y es la encargada de asignar un cmap a la matriz de confusión según el valor del accuracy obtenido. Como parámetro recibe un float que indica el accuracy obtenido. El código de la función *ColorSelector* se adjunta en 2.

Código 2: Función *ColorSelector*.

```
def ColorSelector(accuracy):
  if accuracy > 0.95:
     cmap = 'Greens'
  elif accuracy > 0.9:
     cmap = 'Blues'
  elif accuracy > 0.8:
     cmap = 'Reds'
  else:
     cmap = 'Greys'
  return cmap
```

La segunda función implementada se llama *TrainModel* y es la encargada de entrenar un modelo y obtener las métricas solicitadas. Como parámetros recibe a un objeto que contiene a un modelo de red neuronal (previamente creado con la clase *Model*) y el valor de *patience* que sera utilizado para la implementación de *early stopping patience*, este se encuentre pre asignado en 5. Es importante señalar que el numero de épocas es 1000. El código de la función *TrainModel* se adjunta en 3.

Código 3: Función *TrainModel*.

```
def TrainModel(model, Patience=5):
  # Definición de dispositivo
  device = torch.device('cuda')
  # Se envia al modelo al dispositivo
  model = model.to(device)
  # Definición de la función de perdida
  Criterion = nn.CrossEntropyLoss()
  # Definición del optimizador
  Optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
  # Inicio del tiempo
  StartTime = time.time()
  # Listas
  TrainingLossList = []
  ValidationLossList = []
  EpochsList = []
  # Parametros del early stopping
  PatienceCounter = 0
  BestValidationLoss = float('inf')
  # Bucle sobre el dataset 1000 veces
  for epoch in range(1000):
    # Modelo en modo entrenamiento
    model.train()
    # Parametros de las perdidas
    TrainingLossSumatory = 0.0
    TrainingBatches = 0
    ValidationLossSumatory = 0.0
```

```
    ValidationBatches = 0
    # Entrenamiento para la epoca actual
    for i, data in enumerate(DataloaderTrain, 0):
      # Procesar el lote actual
      TrainingInputs = data["features"].to(device)
      TrainingLabels = data["labels"].to(device)
      # Poner en cero los parametros del gradiente
      Optimizer.zero_grad()
      # Foward + Backward + Optimize
      # Salidas del modelo sobre el conjunto de entrenamiento
      TrainingOutputs = model(TrainingInputs)
      # Perdida del modelo sobre el conjunto de entrenamiento
      TrainingLoss = Criterion(TrainingOutputs, TrainingLabels)
      # Backward
      TrainingLoss.backward()
      # Optimize
      Optimizer.step()
      # Parametros para el calculo del promedio de la perdida en el entrenamiento
      TrainingLossSumatory += TrainingLoss.detach().cpu().item()
      TrainingBatches += 1
    # Calculo y guardado de la perdida promedio en el bucle de entrenamiento
    TrainingLossAverage = TrainingLossSumatory / TrainingBatches
    TrainingLossList.append(TrainingLossAverage)
    # Modelo en evaluación
    model.eval()
    # Se desactiva el computo de gradientes
    with torch.no_grad():
      # Para cada indice y dato en el enumerado del Dataloader del conjunto de validación
      for i, data in enumerate(DataloaderVal):
        # Procesa el lote actual
        ValidationInputs = data["features"].to(device)
        ValidationLabels = data["labels"].to(device)
        # Salidas del modelo sobre el conjunto de validación
        ValidationOutputs = model(ValidationInputs)
        # Perdida  del modelo sobre el conjunto de validación
        ValidationLoss = Criterion(ValidationOutputs, ValidationLabels)
        # Parametros para el calculo del promedio de la perdida en la validación
        ValidationLossSumatory += ValidationLoss.detach().cpu().item()
        ValidationBatches += 1
      # Calculo y guardado de la perdida promedio en el bucle de validación
      ValidationLossAverage = ValidationLossSumatory / ValidationBatches
      ValidationLossList.append(ValidationLossAverage)
    # Guardar la Epoca
    EpochsList.append(epoch)
    # Imprimir epoca, promedio de perdida en entrenamiento y promedio de perdida en
     ↪ validación
    print(f'epoch {epoch}')
```

```
    print(f'Training loss: {TrainingLossAverage:.4f}')
    print(f'Validation loss: {ValidationLossAverage:.4f}')
    # Logica de patience early stopping
    # Si la perdida del promedio del conjunto de validación es menor que la perdida del
     ↪ mejor conjunto de validación
    if ValidationLossAverage < BestValidationLoss:
      # Asignar como mejor perdida a la perdida promedio del conjunto de validación
      BestValidationLoss = ValidationLossAverage
      # Asignar a 0 el contador de paciencia
      PatienceCounter = 0
    # En otro caso
    else:
      # Aumentar en 1 el contador de paciencia
      PatienceCounter += 1
    # Si el contador de paciencia es mayor a la paciencia
    if PatienceCounter > Patience:
      # Imprime mensaje de early stopping
      print(f'Early stoping at epoch {epoch}')
      # Termina el bucle
      break
  # Termino del tiempo
  EndTime = time.time()
```

1.2.1. a)

En la presente sección se creo una red neuronal de 10 neuronas en la capa oculta y función de activación *ReLU*. Además se obtuvieron las métricas solicitadas señaladas previamente. El código implementado para realizar lo anterior se adjunta en 4.

Código 4: Red neuronal con 10 neuronas en la capa oculta y función de activación *ReLU*.

```
model_1 = Model('ReLU', 10, 1)
TrainModel(model_1)
```

Al ejecutar 4 se obtiene:

Tabla 1: Arquitectura de la primera red neuronal.

Capa	Tipo	Características_entrada	Características_salida
Entrada	Linear	64	10
Oculta	ReLU		
Salida	Linear	10	10

Época de parado	Tiempo de entrenamiento
77	4.6587 segundos

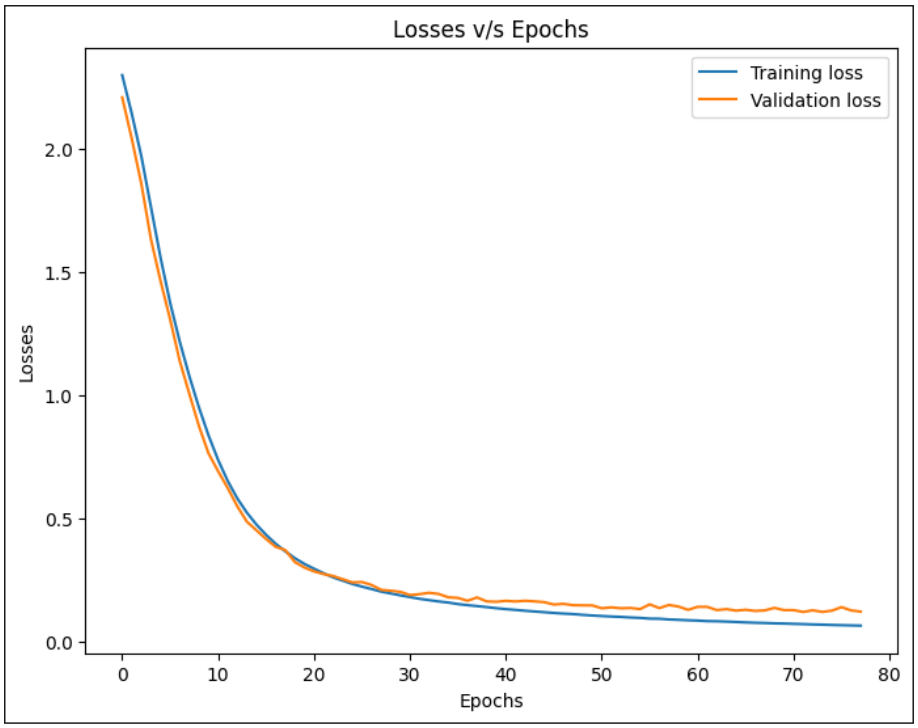


Figura 1: Gráfico perdidas de entrenamiento y validación versus época - Primera red neuronal.

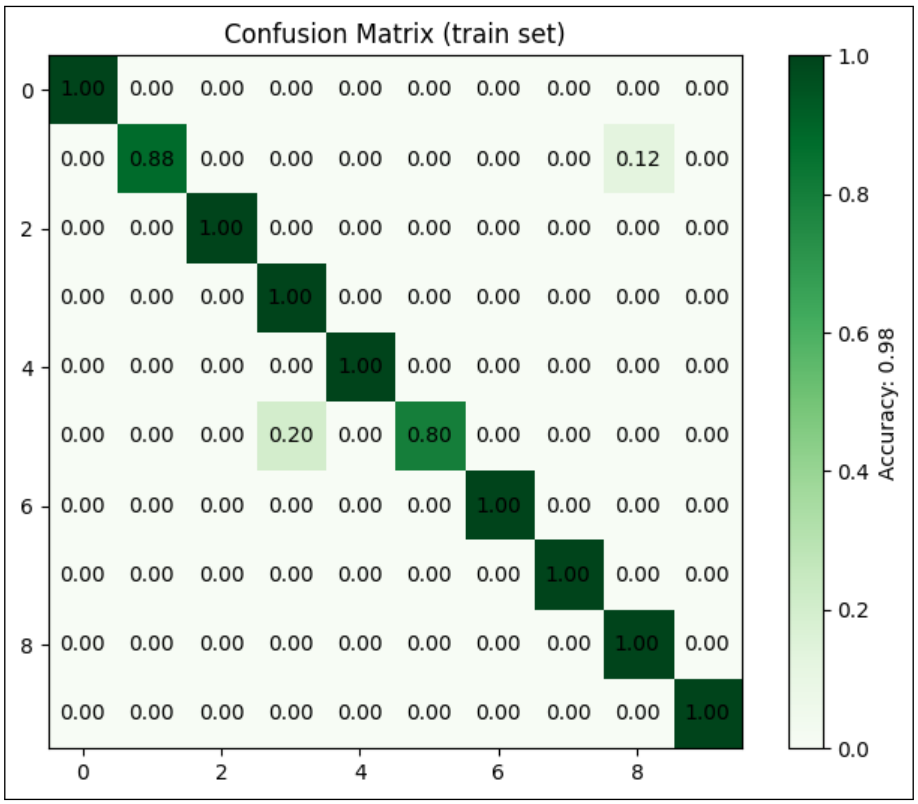


Figura 2: Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Primera red neuronal.

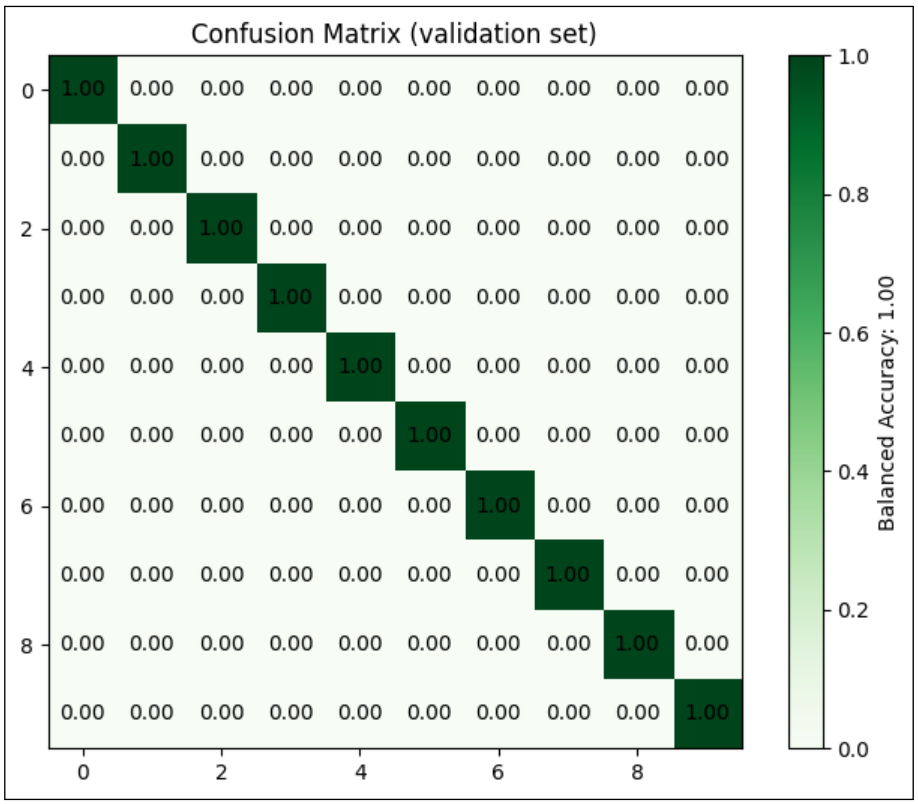


Figura 3: Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Primera red neuronal.

1.2.2. b)

En la presente sección se creo una red neuronal de 40 neuronas en la capa oculta y función de activación *ReLU*. Además se obtuvieron las métricas solicitadas señaladas previamente. El código implementado para realizar lo anterior se adjunta en 5.

Código 5: Red neuronal con 40 neuronas en la capa oculta y función de activación *ReLU*.

```
model_2 = Model('ReLU', 40, 1)
TrainModel(model_2)
```

Al ejecutar 5 se obtiene:

Tabla 2: Arquitectura de la segunda red neuronal.

Capa	Tipo	Características_entrada	Características_salida
Entrada	Linear	64	40
Oculta	ReLU		
Salida	Linear	40	10

Época de parado	Tiempo de entrenamiento
67	5.3368 segundos

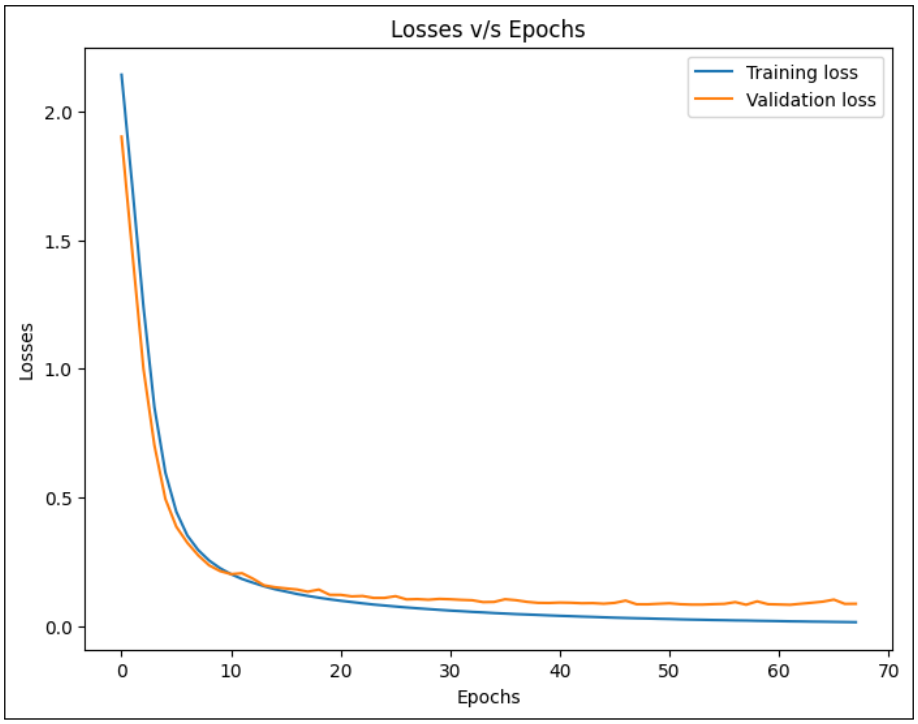


Figura 4: Gráfico perdidas de entrenamiento y validación versus época - Segunda red neuronal.

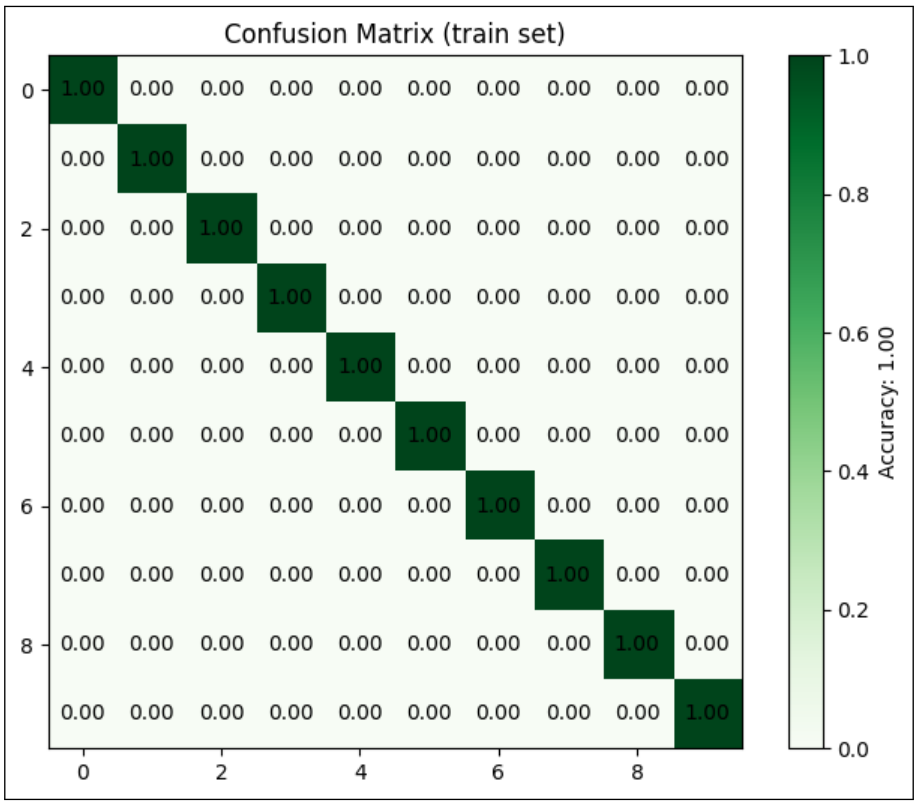


Figura 5: Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Segunda red neuronal.

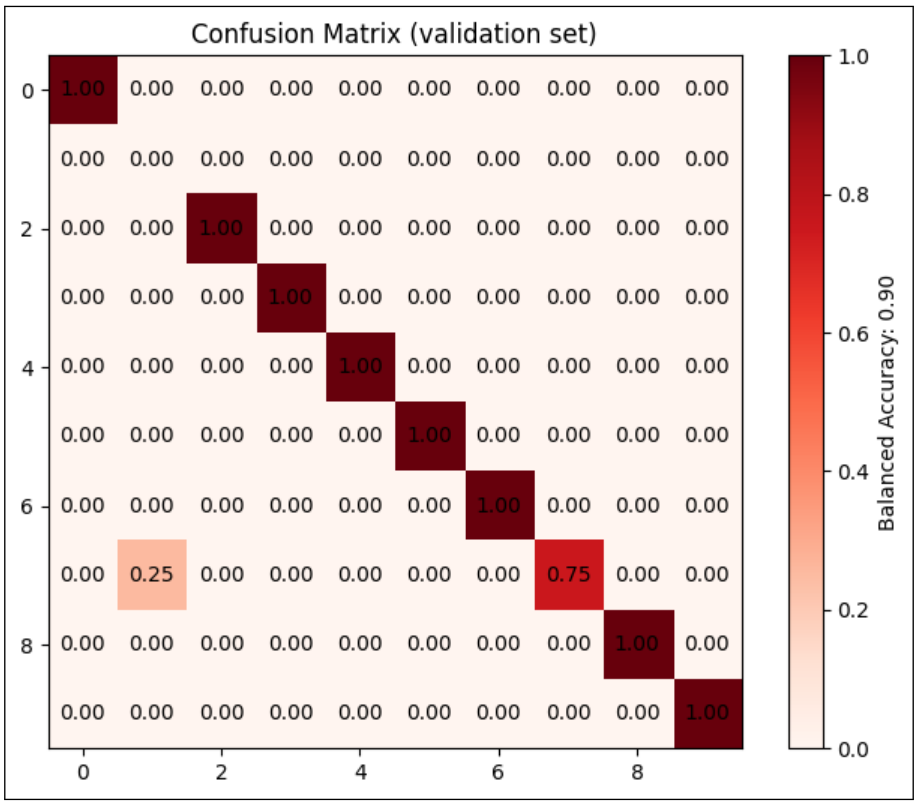


Figura 6: Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Segunda red neuronal.

1.2.3. c)

En la presente sección se creo una red neuronal de 10 neuronas en la capa oculta y función de activación *Tanh*. Además se obtuvieron las métricas solicitadas señaladas previamente. El código implementado para realizar lo anterior se adjunta en 6.

Código 6: Red neuronal con 10 neuronas en la capa oculta y función de activación *Tanh*.

```
model_3 = Model('Tanh', 10, 1)
TrainModel(model_3)
```

Al ejecutar 6 se obtiene:

Tabla 3: Arquitectura de la tercera red neuronal.

Capa	Tipo	Características_entrada	Características_salida
Entrada	Linear	64	10
Oculta	Tanh		
Salida	Linear	10	10

Época de parado	Tiempo de entrenamiento
107	7.2305 segundos

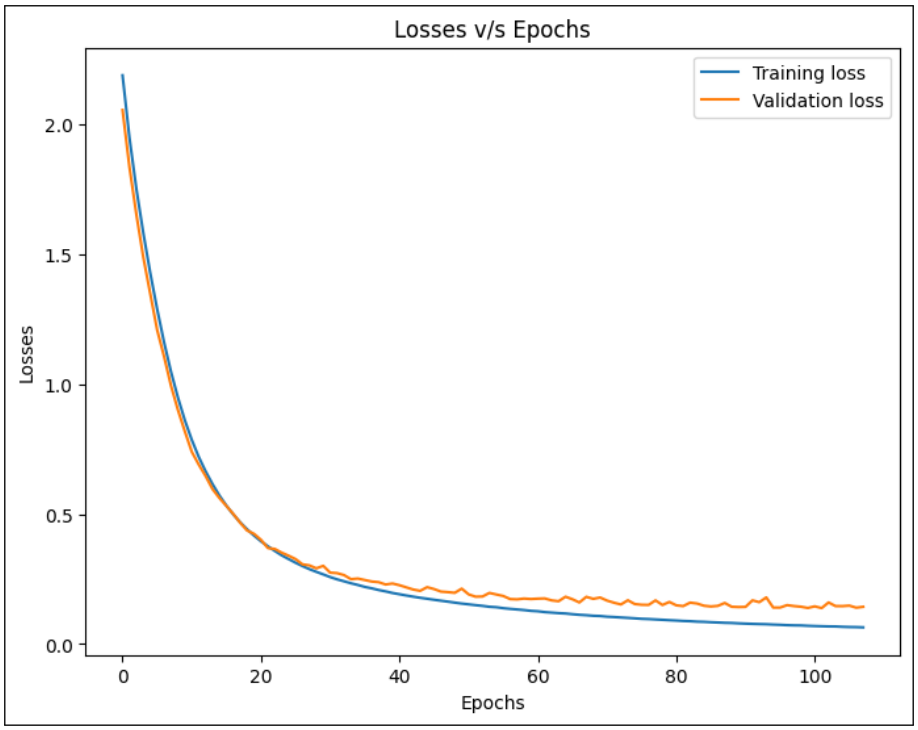


Figura 7: Gráfico perdidas de entrenamiento y validación versus época - Tercera red neuronal.

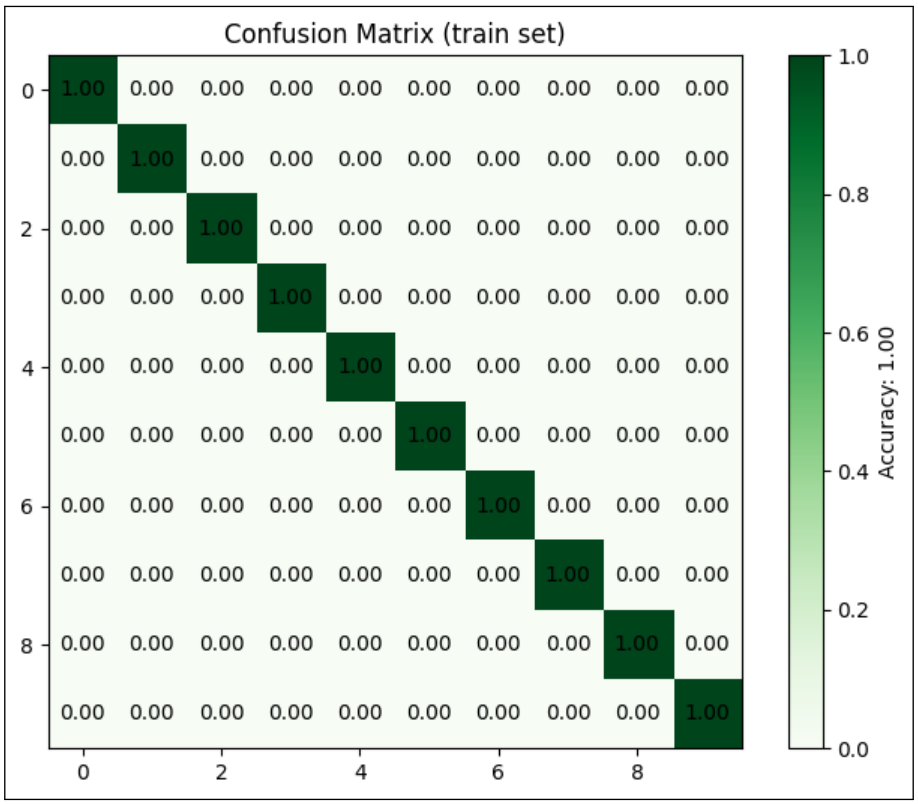


Figura 8: Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Tercera red neuronal.

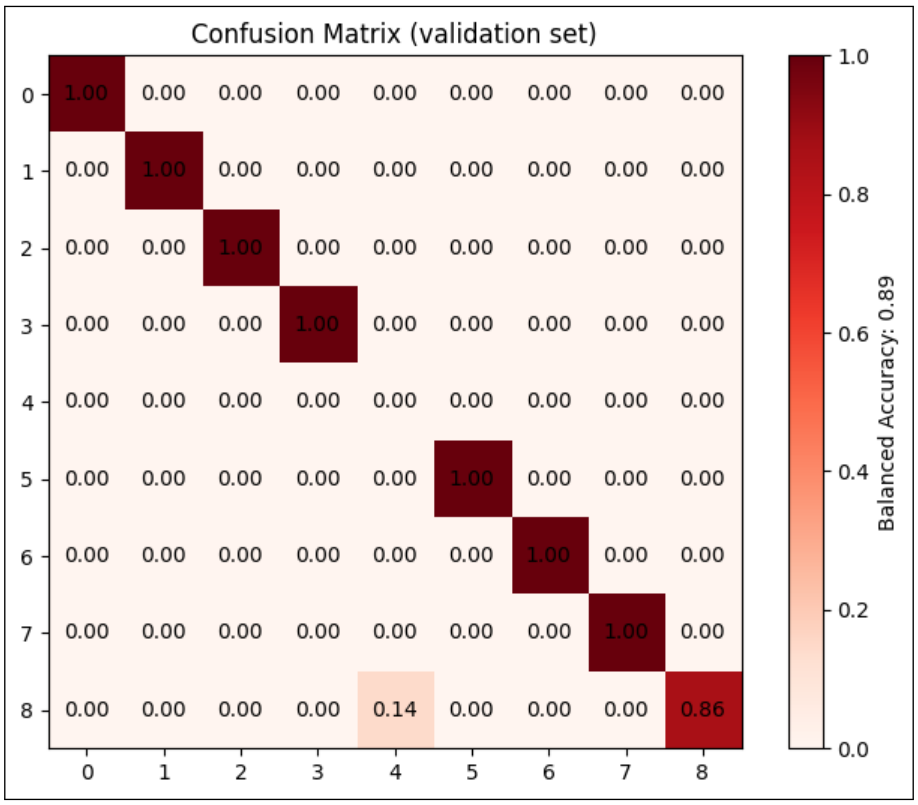


Figura 9: Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Tercera red neuronal.

1.2.4. d)

En la presente sección se creo una red neuronal de 40 neuronas en la capa oculta y función de activación *Tanh*. Además se obtuvieron las métricas solicitadas señaladas previamente. El código implementado para realizar lo anterior se adjunta en 7.

Código 7: Red neuronal con 40 neuronas en la capa oculta y función de activación *Tanh*.

```
model_4 = Model('Tanh', 40, 1)
TrainModel(model_4)
```

Al ejecutar 7 se obtiene:

Tabla 4: Arquitectura de la cuarta red neuronal.

Capa	Tipo	Características_entrada	Características_salida
Entrada	Linear	64	40
Oculta	Tanh		
Salida	Linear	40	10

Época de parado	Tiempo de entrenamiento
49	3.4326 segundos

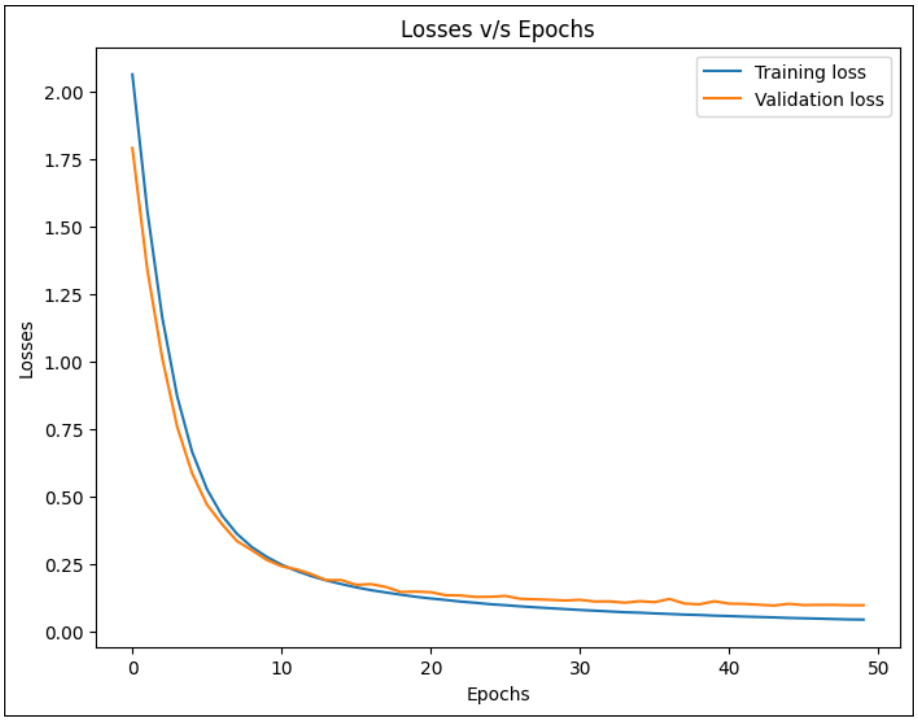


Figura 10: Gráfico perdidas de entrenamiento y validación versus época - Cuarta red neuronal.

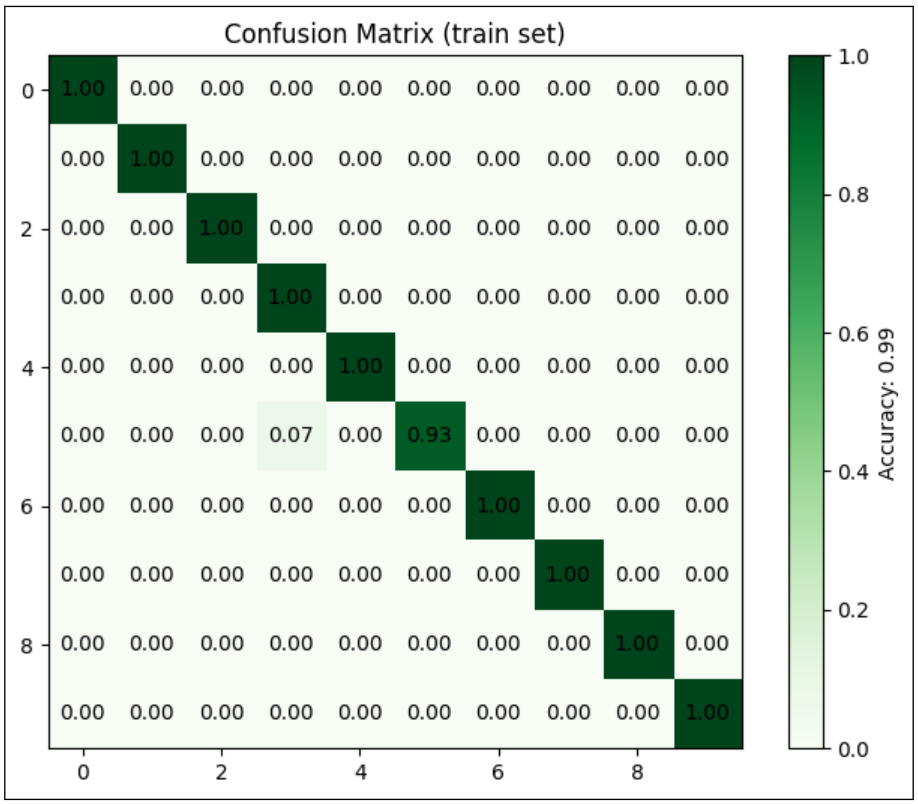


Figura 11: Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Cuarta red neuronal.

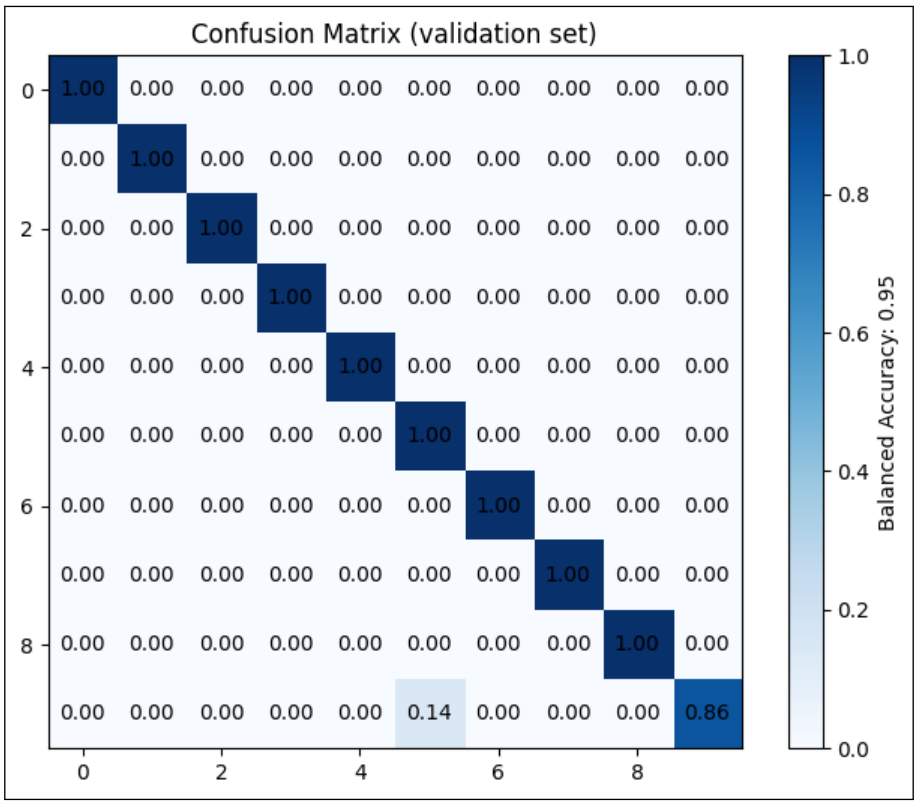


Figura 12: Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Cuarta red neuronal.

1.2.5. e)

En la presente sección se creo una red neuronal de 10 neuronas en la capa oculta, 2 capas ocultas y función de activación *ReLU*. Además se obtuvieron las métricas solicitadas señaladas previamente. El código implementado para realizar lo anterior se adjunta en 8.

Código 8: Red neuronal con 10 neuronas en la capa oculta, 2 capas ocultas y función de activación *ReLU*.

```
model_5 = Model('ReLU', 10, 2)
TrainModel(model_5)
```

Al ejecutar 8 se obtiene:

Tabla 5: Arquitectura de la quinta red neuronal.

Capa	Tipo	Características_entrada	Características_salida
Entrada	Linear	64	10
Oculta	ReLU		
Oculta	ReLU		
Salida	Linear	10	10

Época de parado	Tiempo de entrenamiento
69	4.2721 segundos

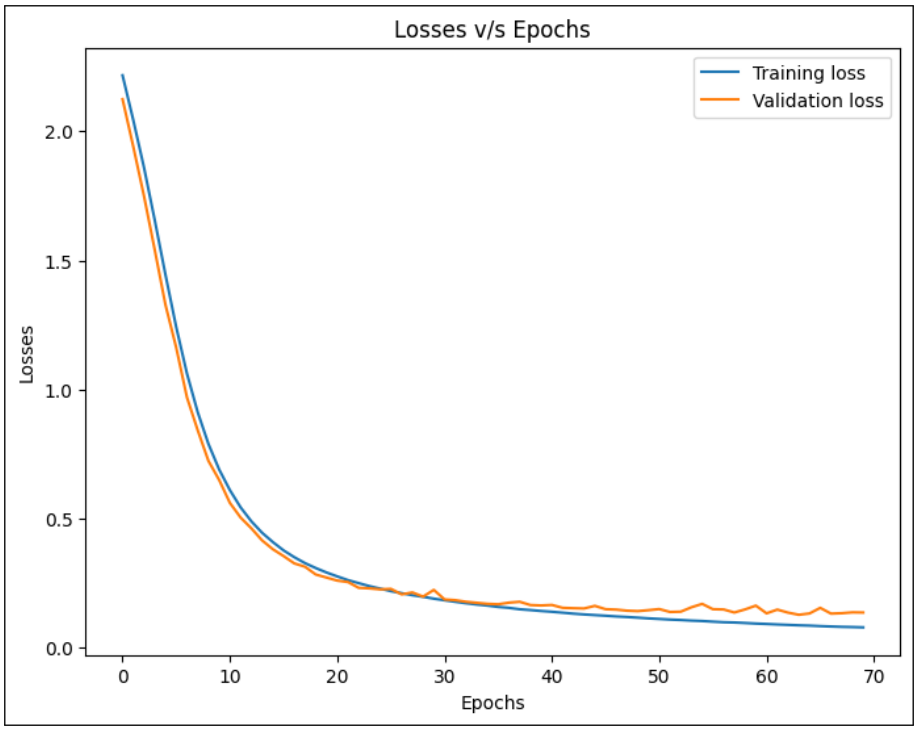


Figura 13: Gráfico perdidas de entrenamiento y validación versus época - Quinta red neuronal.

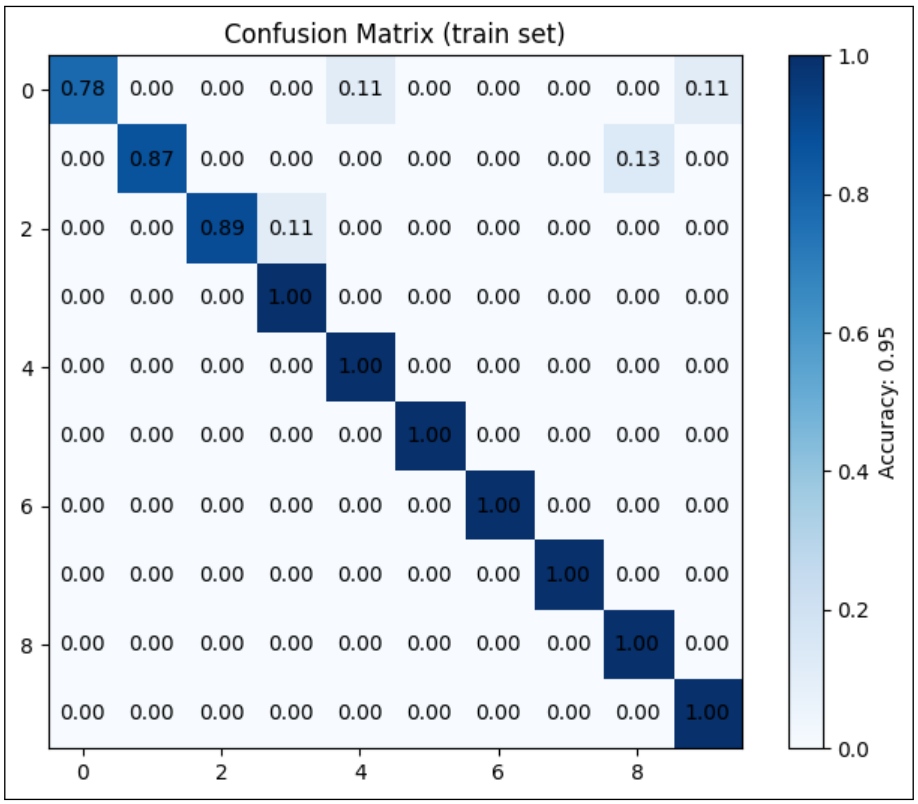


Figura 14: Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Quinta red neuronal.

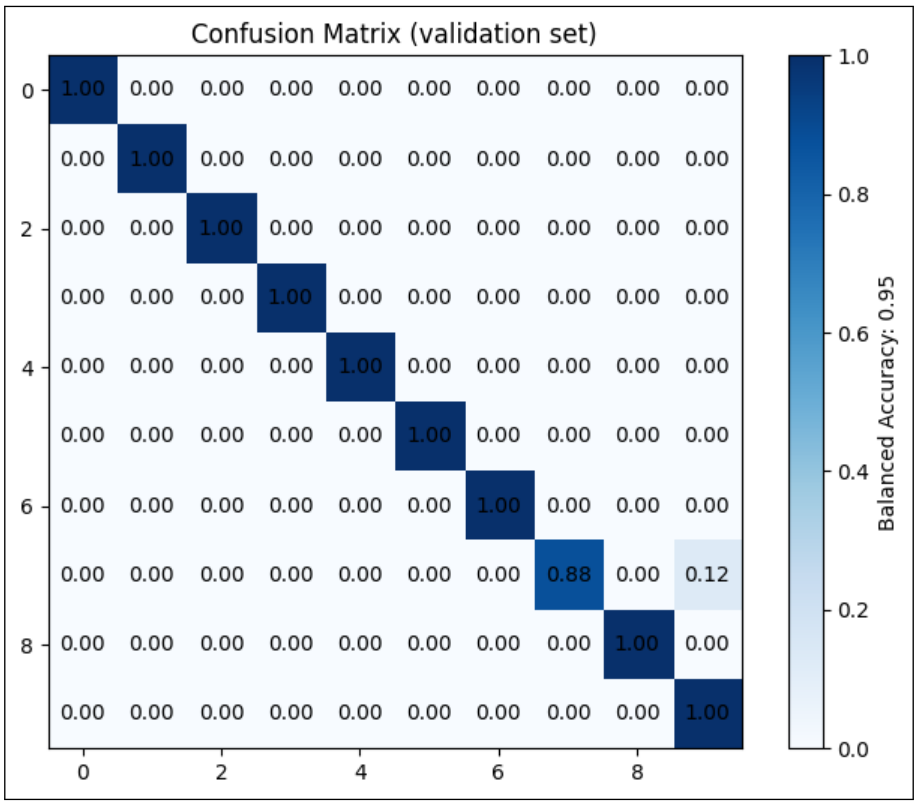


Figura 15: Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Quinta red neuronal.

1.2.6. f)

En la presente sección se creo una red neuronal de 40 neuronas en la capa oculta, 2 capas ocultas y función de activación *ReLU*. Además se obtuvieron las métricas solicitadas señaladas previamente. El código implementado para realizar lo anterior se adjunta en 8.

Código 9: Red neuronal con 40 neuronas en la capa oculta, 2 capas ocultas y función de activación *ReLU*.

```
model_6 = Model('ReLU', 40, 2)
TrainModel(model_6)
```

Al ejecutar 9 se obtiene:

Tabla 6: Arquitectura de la sexta red neuronal.

Capa	Tipo	Características_entrada	Características_salida
Entrada	Linear	64	40
Oculta	ReLU		
Oculta	ReLU		
Salida	Linear	10	40

Época de parado	Tiempo de entrenamiento
72	5.2806 segundos

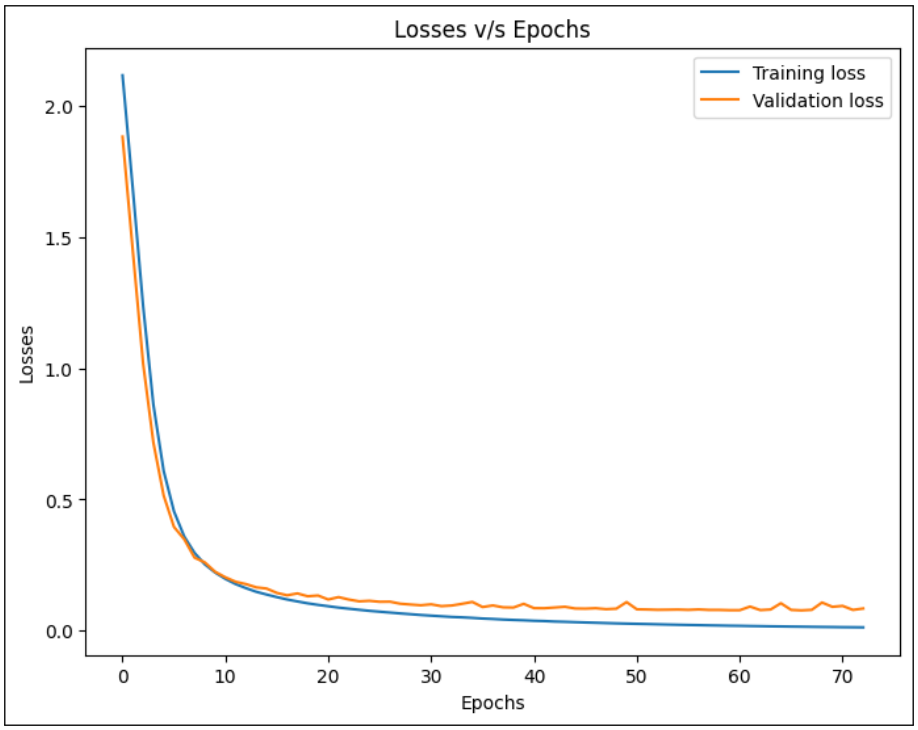


Figura 16: Gráfico perdidas de entrenamiento y validación versus época - Sexta red neuronal.

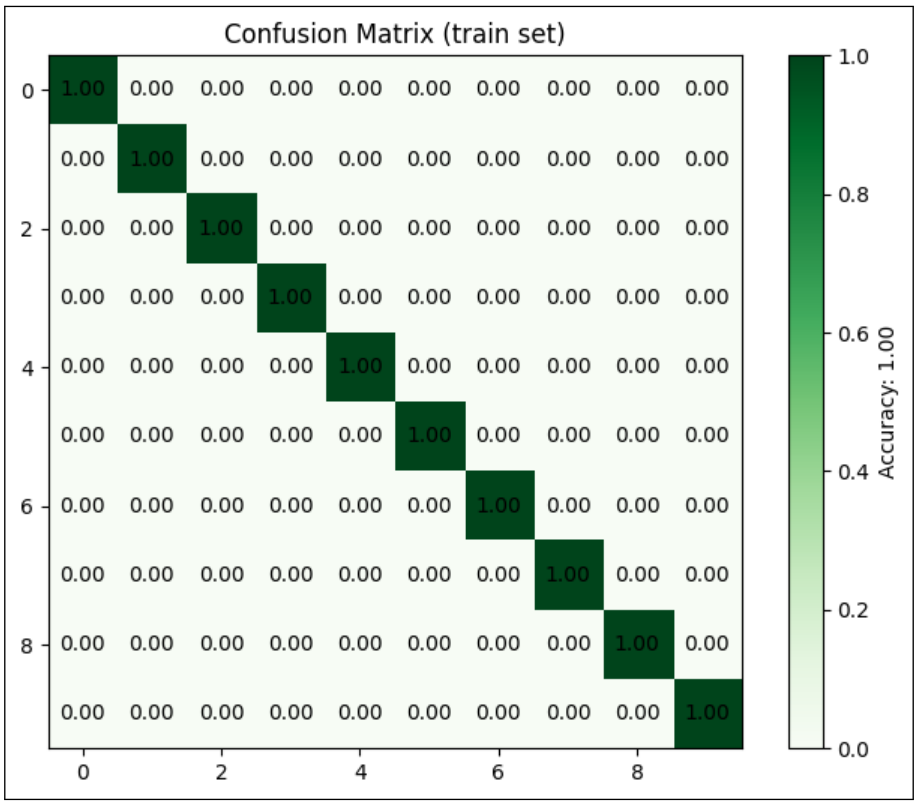


Figura 17: Matriz de confusión normalizada y accuracy sobre conjunto de entrenamiento - Sexta red neuronal.

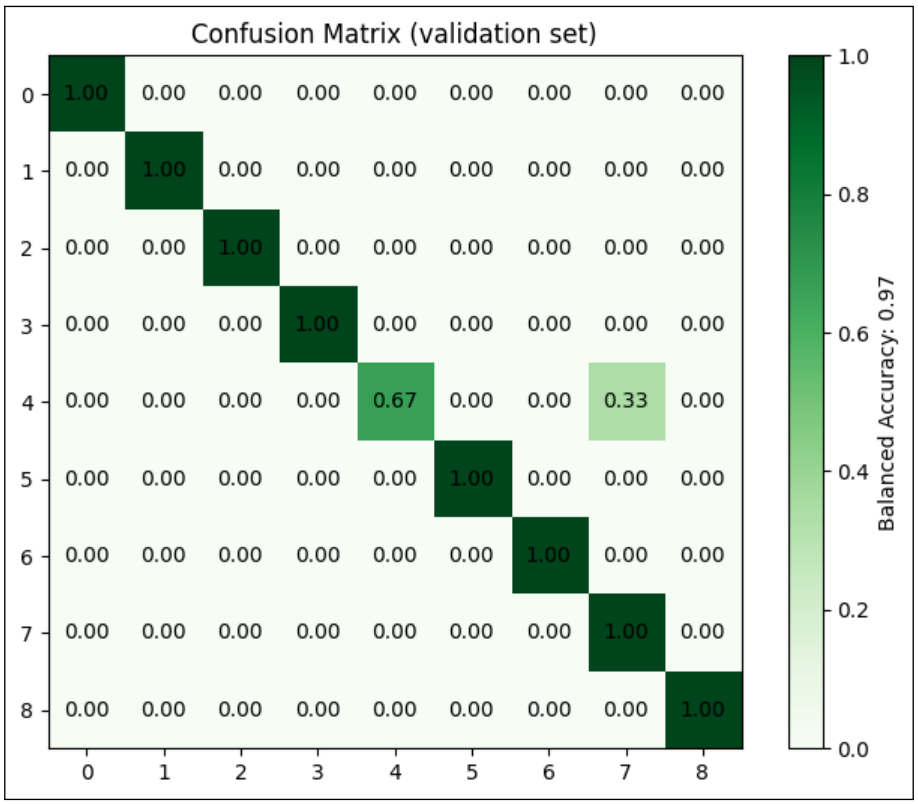


Figura 18: Matriz de confusión normalizada y accuracy normalizado sobre conjunto de validación - Sexta red neuronal.

1.3. 3)

En la presente sección se evaluó, utilizando el conjunto de prueba, la mejor red neuronal de 1.2. El criterio para definir las mejor red neuronal es el accuracy de validación, y debido a que las red de la sección 1.2.1 obtuvo el valor máximo de accuracy sobre el conjunto de validación, dicha red sera la utilizada durante la sección.
Para evaluar al mejor modelo se utilizaron las mismas métricas que en el caso del conjunto de validación, es decir, se utilizo la matriz de confusión normalizada y el accuracy normalizado. Para lo anterior se escribió una función, la cual se adjunta en 10.

Código 10: Función *EvaluateBestModel*

```
def EvaluateBestModel(model):
  device = torch.device('cuda')
  model = model.to(device)

  TestLossList = []
  with torch.no_grad():
     for i, data in enumerate(DataloaderTest):
       # Process the current batch
```

```
9         TestInputs = data["features"].to(device)
10        TestLabels = data["labels"].to(device)
11        TestOutputs = model(TestInputs)
12
13      AccuracyScoreTest = balanced_accuracy_score(TestLabels.detach().cpu().numpy(),
        ↪ torch.argmax(TestOutputs, dim=1).detach().cpu().numpy())
14      ConfusionMatrixTest = confusion_matrix(torch.argmax(TestOutputs, dim=1).detach().
        ↪ cpu().numpy(), TestLabels.detach().cpu().numpy(), normalize='true')
```

Al utilizar 10 sobre el modelo de 1.2.1 se obtiene:

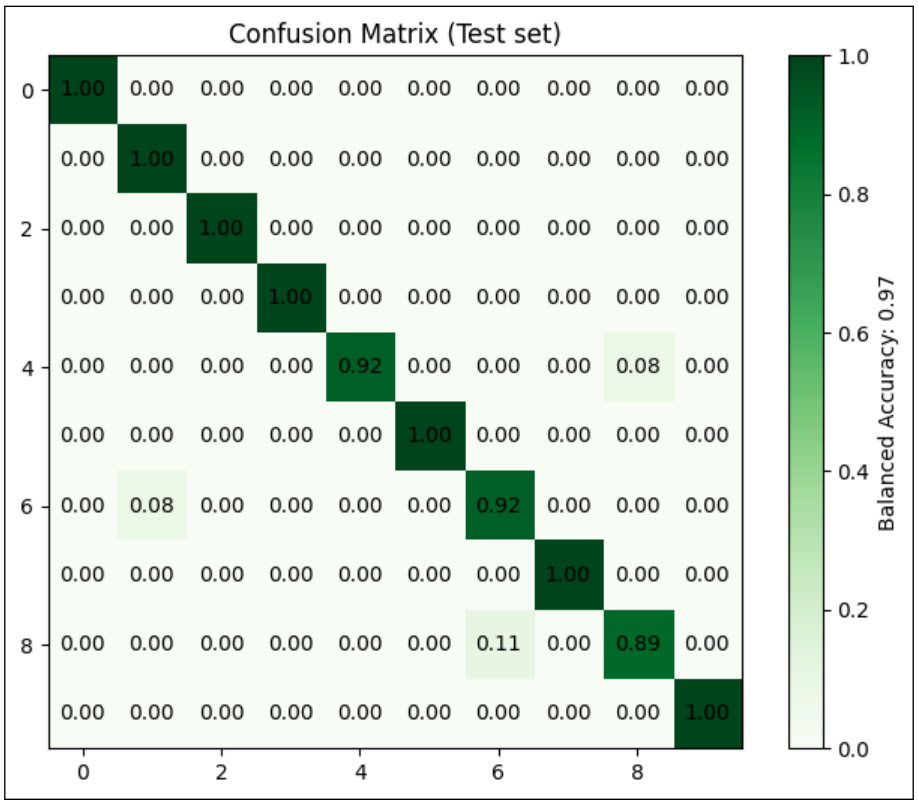


Figura 19: Matriz de confusión normalizada y accuracy normalizado sobre conjunto de prueba - Primera red neuronal.

1.4. 4)

En la presente sección se realizaran diversos análisis respecto a los resultados expuestos en el presente informe.

1.4.1. a)

En la presente sección se ahondara en los efectos de variar la cantidad de neuronas en la capa oculta y como esto afecta el desempeño de la red.

Según lo expuesto en este informe:

Resultados en las métricas No existe una tendencia clara acerca de la cantidad de neuronas óptimas. Es claro que las mejores métricas se obtienen para 1.2.1, pero en el caso de utilizar la misma cantidad de neuronas (10) no se observan resultados similares, pues en 1.2.3 y 1.2.5 se obtienen accuracys normalizados 0.89 y 0.95 respectivamente, los cuales, sin ser malos valores, distan bastante entre ellos y del obtenido en 1.2.1.

Tiempo de entrenamiento Existe una correlación clara en la que utilizar una menor cantidad de neuronas implica un menor tiempo de entrenamiento. Existe un outlier, el caso de las redes que utilizan la función de activación *Tanh*. Al comparar ambas redes, aquella que posee una mayor cantidad de neuronas presenta un tiempo de entrenamiento menor, con un mejor resultado en las métricas sobre el conjunto de validación.

Según la teoría:

Resultados Un mayor numero de neuronas en la capa oculta conlleva una mayor capacidad de la red para capturar características más complejas del conjunto de datos de entrenamiento, sin embargo, esto podría incrementar el riesgo de overfitting, pues la red se ajusta demasiado a los datos del conjunto de entrenamiento y no generaliza para los datos de otros conjuntos.
Un menor numero de neuronas en la capa oculta implica una menor capacidad de capturar características más complejas del conjunto de datos de entrenamiento,y esto podría disminuir el riesgo de overfitting, sin embargo, la red puede no ser capas de capturar las variaciones de los datos complejos.

Tiempo de entrenamiento Un mayor numero de neuronas en la capa oculta conlleva, en general, un mayor tiempo de entrenamiento, debido a más cálculos y parámetros implicados, sumado a que usualmente requiere más épocas para converger.
Un menor numero de neuronas en la capa oculta conlleva, en general, un menor tiempo de entrenamiento, debido a menos cálculos y parámetros implicados, sumado a que usualmente requiere menos épocas para converger.

1.4.2. b)

En la presente sección se ahondara en los efectos de variar la cantidad de capas ocultas y como esto afecta el desempeño de la red.
Según lo expuesto en este informe:

Resultados en las métricas No existe una tendencia clara acerca de la cantidad óptima de capas ocultas, pues al momento de comparar 1.2.1 con 1.2.5 es claro que los resultados expuestos en 1.2.1 son superiores. En cambio, al momento de comparar 1.2.2 con 1.2.6, es claro que los resultados expuestos en 1.2.6 son superiores. Por lo tanto, no existe claridad acerca de una cantidad óptima de capas ocultas.

Tiempo de entrenamiento Existe una tendencia clara en la que utilizar una mayor cantidad de capas ocultas implica un menor tiempo de entrenamiento, pues al comparar

1.2.1 con 1.2.5 y 1.2.2 con 1.2.6, es evidente que los tiempos de entrenamiento de 1.2.5 y 1.2.6 son menores que los de sus contra partes con una capa oculta.

Según la teoría:

Resultados Una mayor cantidad de capas oculta genera una red con la capacidad de capturar características más complejas. Además, la red puede modelar correlaciones más complejas en los datos. Pero, la red es más propensa a realizar overfitting.
Una menor cantidad de capas oculta genera una red con la capacidad de capturar características simples. Además, la red puede modelar correlaciones simples en los datos. Pero, la res es menos propensa a realizar overfitting.

Tiempo de entrenamiento Un mayor numero de capas ocultas conlleva, en general, un mayor tiempo de entrenamiento, debido a más cálculos y parámetros implicados.
Un menor numero de capas ocultas conlleva, en general, un menor tiempo de entrenamiento, debido a menos cálculos y parámetros implicados.

1.4.3. c)

En la presente sección se ahondara en los efectos de variar la función de activación y como esto afecta el desempeño de la red.
Segun lo expuesto en este informe:

Resultados en las métricas No existe una tendencia clara acerca de cual es la función de activación óptima, pues al momento de comparar 1.2.1 con 1.2.3, es claro que los resultados expuestos en 1.2.1 son superiores. En cambio, al momento de comparar 1.2.2 con 1.2.4, es claro que los resultados expuestos en 1.2.4 son superiores. Por lo tanto, no existe claridad acerca de la función de activación óptima.

Tiempo de entrenamiento No existe una tendencia clara acerca de cual función de activación implica un menor tiempo de entrenamiento, pues al comparar los tiempos de entrenamiento de 1.2.1 con 1.2.3, es claro notar que el tiempo de entrenamiento en 1.2.1 es menor, en cambio al comparar los tiempos de entrenamiento de 1.2.2 con 1.2.4, es claro notar que el tiempo de entrenamiento en 1.2.4 es menor.

Según la teoría:

Funciones de activación La función de activación *ReLU* introduce no linealidad, lo cual permite el aprendizaje de relaciones complejas. Además, posee una propagación del gradiente eficiente. Por ultimo, es computacionalmente eficiente.
La función de activación *Tanh* también introduce no linealidad. Pero, posee una propagación del gradiente riesgosa, pues puede producir desvanecimiento del gradiente. Por ultimo, es computacionalmente compleja.

Tiempo de entrenamiento Debido a lo mencionado anteriormente, las redes con función de activación *ReLU* poseen, en general, un menor tiempo de entrenamiento que las redes con función de activación *Tanh*.

1.4.4. d)

En la presente sección se ahondara en el análisis de los tiempos de entrenamiento, matrices de confusión y accuracys sobre el conjunto de validación.

Tiempos de entrenamiento Los tiempos de entrenamiento en orden ascendente, son:

Tabla 7: Tiempos de entrenamiento de los modelos.

Modelo	Tiempo de entrenamiento
Cuarto	3.4326
Quinto	4.2721
Primero	4.6587
Sexto	5.2806
Segundo	5.3368
Tercero	7.2305

Respecto a 7, es claro que no existe una tendencia en cuanto a la arquitectura de los modelo y el tiempo de entrenamiento.
Aunque, el tiempo de entrenamiento parece tener algo de importancia respecto a los resultados obtenidos sobre el conjunto de validación, pues tiempos de entrenamiento extensos parecen implicar peores resultados sobre dicho conjunto, en particular, el tercer modelo.

Matrices de confusión sobre el conjunto de validación Análisis por red:

Primera Red Matriz de confusión perfecta. Posee únicamente valores nulos fuera de la diagonal.

Segunda Red Ausencia de instancias de la clase 1 en el conjunto *ValidationLabels*. Un 25 % de las instancias de la clase 7 fueron erróneamente clasificadas en clase 1.

Tercera Red Ausencia de instancias de las clases 4 y 9 en el conjunto de *ValidationLabels*. Un 14 % de las instancias de la clase 8 fueron erróneamente clasificadas en clase 4.

Cuarta Red Valores nulos fuera de la diagonal para todas las clases menos la clase 9. En dicha clase, un 14 % de las instancias fueron erróneamente clasificadas en clase 5.

Quinta Red Valores nulos fuera de la diagonal para todas las clases menos la clase 7. En dicha clase, un 12 % de las instancias fueron erróneamente clasificadas en clase 9.

Sexta Red Ausencia de la clase 9. Valores nulos fuera de la diagonal para todas las clases menos la clase 4. En dicha clase, un 33 % de las instancias fueron erróneamente clasificadas en clase 7.

Accuracys sobre el conjunto de validación Respecto a los accuracys normalizados obtenidos sobre el conjunto de validación, es claro que los valores obtenidos en todas las redes neuronales son buenos, pues el mínimo fue 0.89, el cual no es un valor bajo y no dista mucho del máximo 1.00.
Además, cuatro de los accuracys obtenidos, fueron mayores a 0.95, lo cual demuestra que las redes neuronales implementadas son muy buenas para este problema de clasificación. Aunque existen dos casos en que los accuracys obtenidos fueron menores a 0.95, en dichos casos se obtuvieron accuracys de 0.89 y 0.90, los cuales continúan siendo valores elevados de accuracy. Además, el bajo valor de estos puede deberse a la ausencia de clases en el conjunto de validación utilizado.

1.4.5. e)

En la presente sección se ahondara en la comparación y análisis de los resultados obtenidos sobre el conjunto de validación y prueba utilizando el modelo 1.2.1.

Matrices de confusión Como se ha mencionado previamente, la matriz de confusión del modelo 1.2.1 sobre el conjunto de validación es perfecta. Aun así, la matriz de confusión obtenida sobre el conjunto de prueba no se queda muy atrás, pues esta: Posee únicamente 3 valores distintos a 1.00 en la diagonal, es decir, la red comete errores de clasificación en solo 3 clases. Dichas clases son:

- La clase 4, donde un 8 % de instancias fueron erróneamente clasificadas en clase 8.
- La clase 6, donde un 8 % de instancias fueron erróneamente clasificadas en clase 2.
- La clase 8, donde un 11 % de instancias fueron erróneamente clasificadas en clase 6.

Accuracys Como se ha mencionado previamente, el valor del accuracy normalizado del modelo 1.2.1 sobre el conjunto de validación es perfecto, es decir, tiene un valor de 1.00. Aun así, el valor del accuracy normalizado sobre el conjunto de prueba no se queda muy atrás. pues este es 0.97, lo cual implica una diferencia de 3 centésimas.

Es por lo anterior que es posible afirmar que la red neuronal entrenada en 1.2.1 es robusta y permite una correcta clasificación de datos con los cuales no fue entrenado.

2. Conclusiones

En la presente sección se presentaran las conclusiones generales del informe.

2.1. Aprendizajes

A continuación, se describen mis aprendizajes.

Balanced Accuracy He aprendido acerca del accuracy normalizado, el cual es una métrica que muy útil al momento de comparar clasificadores en los que el conjunto de datos a utilizar no este balanceado.

Redes Neuronales He aprendido a implementar redes neuronales con arquitecturas simples en python mediante el uso de *pytorch*. Además, he aprendido acerca de las implicancias teóricas y practicas de modificar la cantidad de neuronas en la capa oculta, la cantidad de capas ocultas y la función de activación de esta. Así mismo, he aprendido acerca de la aleatoriedad de los resultados de una red neuronal. Con lo anterior me refiero a que los valores de las métricas obtenidas en una red neuronal con una arquitectura particular pueden variar cada vez que se entrene la red.

2.2. Dificultades

- Desconocimiento acerca de la necesidad de «enviar» los datos utilizados en una red neuronal nuevamente a la cpu para utilizar estos.
- Desconocimiento del uso de *balanced_accuracy_score*.
- Desconocimiento de la lógica de *early patience stopping*.

2.3. Evaluación de resultados

Los resultados obtenidos son ambiguos según los que se esperan en función de la teoría.
Según se expuso en 1.4 y según la implementación utilizada:
Las redes neuronales con un mayor numero de neuronas en la capa oculta debiesen haber obtenido mejores métricas que aquellos con menor numero de neuronas en la capa oculta, aunque se cumple que las primeras tardan mas tiempo en entrenarse.
Respecto a la cantidad de capas ocultas a utilizar, los resultados son los esperados, pues el conjunto de datos utilizados no es tan complejo, por lo que utilizar más de una capa oculta produce un peor desempeño debido al overfitting.
Respecto a las funciones de activación de la capa oculta, los resultados son los esperados, pues en general, *ReLU* es superior a *Tanh*, exceptuando casos específicos.
Finalmente, la aleatoriedad de los resultados obtenidos al momento de entrenar múltiples veces una red neuronal particular, se corresponden con la teoría. En particular con la implementación realizada utilizando *pytoch*.

Anexo A. Código

Código A.1: Código de la tarea.

```
# -*- coding: utf-8 -*-
"""Tarea 4 – Redes Neuronales Artificiales EL4106 - Diego Vega.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1Jvg8lyOgZq_QmEVyN3bkIOVkguWRoovM
"""

import pandas as pd
import torch
import torch.nn as nn
import numpy as np
import time
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay,
    ↪ balanced_accuracy_score
from matplotlib import pyplot as plt

"""# 1)

## Subir datasets de dígitos (train)
"""

from google.colab import files
uploaded = files.upload()  # 1_digits_train.txt
!ls

from google.colab import files
uploaded = files.upload()  # 1_digits_test.txt
!ls

"""## Leer dataset de dígitos"""

column_names = ["feat" + str(i) for i in range(64)]
column_names.append("class")

df_train_val = pd.read_csv('1_digits_train.txt', names = column_names)
df_train_val

df_test = pd.read_csv('1_digits_test.txt', names = column_names)
```

```
df_test

df_train, df_val = train_test_split(df_train_val, test_size = 0.3, random_state = 10)

scaler = StandardScaler().fit(df_train.iloc[:,0:64])
df_train.iloc[:,0:64] = scaler.transform(df_train.iloc[:,0:64])
df_val.iloc[:,0:64] = scaler.transform(df_val.iloc[:,0:64])
df_test.iloc[:,0:64] = scaler.transform(df_test.iloc[:,0:64])

df_train

"""## Crear datasets y dataloaders para pytorch (train)"""

# Crear datasets
feats_train = df_train.to_numpy()[:,0:64].astype(np.float32)
labels_train = df_train.to_numpy()[:,64].astype(int)
dataset_train = [ {"features":feats_train[i,:], "labels":labels_train[i]} for i in range(
    ↪ feats_train.shape[0]) ]

feats_val = df_val.to_numpy()[:,0:64].astype(np.float32)
labels_val = df_val.to_numpy()[:,64].astype(int)
dataset_val = [ {"features":feats_val[i,:], "labels":labels_val[i]} for i in range(feats_val.shape
    ↪ [0]) ]

feats_test = df_test.to_numpy()[:,0:64].astype(np.float32)
labels_test = df_test.to_numpy()[:,64].astype(int)
dataset_test = [ {"features":feats_test[i,:], "labels":labels_test[i]} for i in range(feats_test.
    ↪ shape[0]) ]

dataset_train[0]["features"]

# Crear dataloaders
DataloaderTrain = torch.utils.data.DataLoader(dataset_train, batch_size=128, shuffle=True
    ↪ , num_workers=0)
DataloaderVal = torch.utils.data.DataLoader(dataset_val, batch_size=128, shuffle=True,
    ↪ num_workers=0)
DataloaderTest = torch.utils.data.DataLoader(dataset_test, batch_size=128, shuffle=True,
    ↪ num_workers=0)

"""## Crear modelo"""

model = nn.Sequential(
        nn.Linear(64, 10),
        nn.ReLU(),
        nn.Linear(10,10)
      )

```

```
device = torch.device('cuda')

model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

"""## Entrenamiento"""

start = time.time()

# loop over the dataset multiple times
for epoch in range(250):
  model.train()
  # Train on the current epoch
  for i, data in enumerate(DataloaderTrain, 0):
    # Process the current batch
    inputs = data["features"].to(device)
    labels = data["labels"].to(device)
    # zero the parameter gradients
    optimizer.zero_grad()
    # forward + backward + optimize
    outputs = model(inputs)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()

  model.eval()
  with torch.no_grad():
    # To do: compute validation loss for current epoch
    pass

  # To do: print train/validation loss for the current epoch
  print('epoch %d' % (epoch))

end = time.time()
print('Finished Training, total time %f seconds' % (end - start))

"""# 2)

"""

class Model(nn.Module):
  def __init__(self, activation_layer, hidden_size, n_hidden_layers, input_size=64,
    ↪ output_size=10):
    super(Model, self).__init__()
    self.input_layer = nn.Linear(input_size, hidden_size)
```

```
129     if n_hidden_layers == 2:
130       self.hidden_layer = nn.Sequential(nn.ReLU() if activation_layer == 'ReLU' else nn.
        ↪ Tanh() if activation_layer == 'Tanh' else None,
131                                         nn.ReLU() if activation_layer == 'ReLU' else nn.Tanh() if
        ↪ activation_layer == 'Tanh' else None)
132     elif n_hidden_layers == 1:
133       self.hidden_layer = nn.ReLU() if activation_layer == 'ReLU' else nn.Tanh() if
        ↪ activation_layer == 'Tanh' else None
134     self.output_layer = nn.Linear(hidden_size, output_size)
135     self.n_hidden_layers = n_hidden_layers
136
137   def forward(self, x):
138     out = self.input_layer(x)
139     if self.hidden_layer is not None:
140       if self.n_hidden_layers == 2:
141         for layer in self.hidden_layer:
142           out = layer(out)
143       elif self.n_hidden_layers == 1:
144         out = self.hidden_layer(out)
145     out = self.output_layer(out)
146     return out
147
148 def ColorSelector(accuracy):
149   if accuracy > 0.95:
150       cmap = 'Greens'
151   elif accuracy > 0.9:
152       cmap = 'Blues'
153   elif accuracy > 0.8:
154       cmap = 'Reds'
155   else:
156       cmap = 'Greys'
157   return cmap
158
159 def TrainModel(model, Patience=5):
160   # Definición de dispositivo
161   device = torch.device('cuda')
162   # Se envia al modelo al dispositivo
163   model = model.to(device)
164   # Definición de la función de perdida
165   Criterion = nn.CrossEntropyLoss()
166   # Definición del optimizador
167   Optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
168   # Inicio del tiempo
169   StartTime = time.time()
170   # Listas
171   TrainingLossList = []
172   ValidationLossList = []
```

```
EpochsList = []
# Parametros del early stopping
PatienceCounter = 0
BestValidationLoss = float('inf')
# Bucle sobre el dataset 1000 veces
for epoch in range(1000):
  # Modelo en modo entrenamiento
  model.train()
  # Parametros de las perdidas
  TrainingLossSumatory = 0.0
  TrainingBatches = 0
  ValidationLossSumatory = 0.0
  ValidationBatches = 0
  # Entrenamiento para la epoca actual
  for i, data in enumerate(DataloaderTrain, 0):
    # Procesar el lote actual
    TrainingInputs = data["features"].to(device)
    TrainingLabels = data["labels"].to(device)
    # Poner en cero los parametros del gradiente
    Optimizer.zero_grad()
    # Foward + Backward + Optimize
    # Salidas del modelo sobre el conjunto de entrenamiento
    TrainingOutputs = model(TrainingInputs)
    # Perdida del modelo sobre el conjunto de entrenamiento
    TrainingLoss = Criterion(TrainingOutputs, TrainingLabels)
    # Backward
    TrainingLoss.backward()
    # Optimize
    Optimizer.step()
    # Parametros para el calculo del promedio de la perdida en el entrenamiento
    TrainingLossSumatory += TrainingLoss.detach().cpu().item()
    TrainingBatches += 1
  # Calculo y guardado de la perdida promedio en el bucle de entrenamiento
  TrainingLossAverage = TrainingLossSumatory / TrainingBatches
  TrainingLossList.append(TrainingLossAverage)
  # Modelo en evaluación
  model.eval()
  # Se desactiva el computo de gradientes
  with torch.no_grad():
    # Para cada indice y dato en el enumerado del Dataloader del conjunto de validación
    for i, data in enumerate(DataloaderVal):
      # Procesa el lote actual
      ValidationInputs = data["features"].to(device)
      ValidationLabels = data["labels"].to(device)
      # Salidas del modelo sobre el conjunto de validación
      ValidationOutputs = model(ValidationInputs)
      # Perdida  del modelo sobre el conjunto de validación
```

```
220         ValidationLoss = Criterion(ValidationOutputs, ValidationLabels)
221         # Parametros para el calculo del promedio de la perdida en la validación
222         ValidationLossSumatory += ValidationLoss.detach().cpu().item()
223         ValidationBatches += 1
224       # Calculo y guardado de la perdida promedio en el bucle de validación
225       ValidationLossAverage = ValidationLossSumatory / ValidationBatches
226       ValidationLossList.append(ValidationLossAverage)
227     # Guardar la Epoca
228     EpochsList.append(epoch)
229     # Imprimir epoca, promedio de perdida en entrenamiento y promedio de perdida en
        ↪ validación
230     print(f'epoch {epoch}')
231     print(f'Training loss: {TrainingLossAverage:.4f}')
232     print(f'Validation loss: {ValidationLossAverage:.4f}')
233     # Logica de patience early stopping
234     # Si la perdida del promedio del conjunto de validación es menor que la perdida del
        ↪ mejor conjunto de validación
235     if ValidationLossAverage < BestValidationLoss:
236       # Asignar como mejor perdida a la perdida promedio del conjunto de validación
237       BestValidationLoss = ValidationLossAverage
238       # Asignar a 0 el contador de paciencia
239       PatienceCounter = 0
240     # En otro caso
241     else:
242       # Aumentar en 1 el contador de paciencia
243       PatienceCounter += 1
244     # Si el contador de paciencia es mayor a la paciencia
245     if PatienceCounter > Patience:
246       # Imprime mensaje de early stopping
247       print(f'Early stoping at epoch {epoch}')
248       # Termina el bucle
249       break
250   # Termino del tiempo
251   EndTime = time.time()
252   # Imprime mensaje de termino
253   print('Finished Training, total time %f seconds' % (EndTime - StartTime))
254   # Grafico de perdidas versus epocas
255   plt.figure(figsize=(8,6))
256   plt.plot(EpochsList, TrainingLossList, label='Training loss')
257   plt.plot(EpochsList, ValidationLossList, label='Validation loss')
258   plt.title('Losses v/s Epochs')
259   plt.xlabel('Epochs')
260   plt.ylabel('Losses')
261   plt.legend()
262   plt.show()
263   # Matriz de confusión normalizada y Accuracy sobre el conjunto de entrenamiento
```

```
264   AccuracyScoreTraining = accuracy_score(TrainingLabels.detach().cpu().numpy(), torch.
        ↪ argmax(TrainingOutputs, dim=1).detach().cpu().numpy())
265   ConfusionMatrixTraining = confusion_matrix(torch.argmax(TrainingOutputs, dim=1).
        ↪ detach().cpu().numpy(), TrainingLabels.detach().cpu().numpy(), normalize='true')
266   plt.figure(figsize=(8,6))
267   plt.imshow(ConfusionMatrixTraining, cmap=plt.get_cmap(ColorSelector(
        ↪ AccuracyScoreTraining)))
268   for i in range(ConfusionMatrixTraining.shape[0]):
269       for j in range(ConfusionMatrixTraining.shape[1]):
270           plt.text(j, i, format(ConfusionMatrixTraining[i,j], ".2f"), ha="center", va="center",
        ↪ color="black")
271   plt.colorbar(label=f'Accuracy: {AccuracyScoreTraining:.2f}')
272   plt.title(f'Confusion Matrix (train set)')
273   plt.show()
274   # Matriz de confusión normalizada y Accurac normalizado sobre el conjunto de validación
275   AccuracyScoreValidation = balanced_accuracy_score(ValidationLabels.detach().cpu().
        ↪ numpy(), torch.argmax(ValidationOutputs, dim=1).detach().cpu().numpy())
276   ConfusionMatrixValidation = confusion_matrix(torch.argmax(ValidationOutputs, dim=1).
        ↪ detach().cpu().numpy(), ValidationLabels.detach().cpu().numpy(), normalize='true')
277   plt.figure(figsize=(8,6))
278   plt.imshow(ConfusionMatrixValidation, cmap=plt.get_cmap(ColorSelector(
        ↪ AccuracyScoreValidation)))
279   for i in range(ConfusionMatrixValidation.shape[0]):
280       for j in range(ConfusionMatrixValidation.shape[1]):
281           plt.text(j, i, format(ConfusionMatrixValidation[i, j], ".2f"), ha="center", va="center",
        ↪ color="black")
282   plt.colorbar(label=f'Balanced Accuracy: {AccuracyScoreValidation:.2f}')
283   plt.title(f'Confusion Matrix (validation set)')
284   plt.show()
285
286 """## a)"""
287
288 model_1 = Model('ReLU', 10, 1)
289 print(model_1)
290 TrainModel(model_1)
291
292 """## b)
293
294 """
295
296 model_2 = Model('ReLU', 40, 1)
297 print(model_2)
298 TrainModel(model_2)
299
300 """## c)
301
302 """
```

```
303
304 model_3 = Model('Tanh', 10, 1)
305 print(model_3)
306 TrainModel(model_3)
307
308 """## d)"""
309
310 model_4 = Model('Tanh', 40, 1)
311 print(model_4)
312 TrainModel(model_4)
313
314 """## e)"""
315
316 model_5 = Model('ReLU', 10, 2)
317 print(model_5)
318 TrainModel(model_5)
319
320 """## f)"""
321
322 model_6 = Model('ReLU', 40, 2)
323 print(model_6)
324 TrainModel(model_6)
325
326 """# 3)"""
327
328 def EvaluateBestModel(model):
329   device = torch.device('cuda')
330   model = model.to(device)
331
332   TestLossList = []
333   with torch.no_grad():
334     for i, data in enumerate(DataloaderTest):
335       # Process the current batch
336       TestInputs = data["features"].to(device)
337       TestLabels = data["labels"].to(device)
338       TestOutputs = model(TestInputs)
339
340     AccuracyScoreTest = balanced_accuracy_score(TestLabels.detach().cpu().numpy(),
    ↪ torch.argmax(TestOutputs, dim=1).detach().cpu().numpy())
341     ConfusionMatrixTest = confusion_matrix(torch.argmax(TestOutputs, dim=1).detach().
    ↪ cpu().numpy(), TestLabels.detach().cpu().numpy(), normalize='true')
342     plt.figure(figsize=(8,6))
343     plt.imshow(ConfusionMatrixTest, cmap='Greens')
344     for i in range(ConfusionMatrixTest.shape[0]):
345         for j in range(ConfusionMatrixTest.shape[1]):
346             plt.text(j, i, format(ConfusionMatrixTest[i, j], ".2f"), ha="center", va="center",
    ↪ color="black")
```

```
347         plt.colorbar(label=f'Balanced Accuracy: {AccuracyScoreTest:.2f}')
348         plt.title(f'Confusion Matrix (Test set)')
349         plt.show()
350 
351 EvaluateBestModel(model_1)
```